

Project3 - report

2022036208 인범진

Project 3 - Memory-Mapped Files in xv6

Design

- 본 과제의 목표는 xv6에 `mmap` / `munmap` 시스템 콜을 구현하여 파일을 프로세스의 주소 공간에 매핑하는 기능을 추가하는 것이다
- 구현은 명세가 제시한 네 단계로 나누어 진행했다

Memory Layout

- mmap 영역은 TRAPFRAME 바로 아래에서 아래 방향으로 자란다
- 이때 heap(`p->sz`)과 겹치지 않도록 하며, 두 영역은 서로를 침범하지 않는다

```
TRAMPOLINE
TRAPFRAME      ← mmap 영역의 최상단 경계
[mmap region N]
[mmap region N-1]
...
[mmap region 1]
[unused]
heap (p->sz까지) ← sbrk로 늘어남
stack
text / data
0
```

- `sbrk` 는 heap 영역을 줄이거나, 늘리는 함수로 `growproc()` 을 호출한다
- `growproc()` 은 `sz + n > TRAPFRAME` 이면 거부하여 mmap 영역을 침범하지 못하도록 한다
- 이때 mmap영역은 TRAPFRAME바로 아래에서 자라므로, mmap 영역의 최상단이 항상 TRAPFRAME이다
 - 처음에는 `sbrk` 와 `growproc()` 에 대한 이해만 있어, heap이 자라는 경우에 mmap 영역을 침범할 수 있겠다고 생각했다
 - 그러나 mmap에서 아래로 자라나는 경우에 heap의 최상단보다는 항상 위에 있도록 하면 문제는 발생하지 않는다
- 따라서 `sys_mmap` 은 할당 주소가 `p->sz` 이하로 내려가면 거부한다
- 두 영역은 서로 다른 방향에서 자라며 독립적으로 관리된다

VMA (Virtual Memory Area) 자료구조

- 각 mmap 매핑의 메타데이터를 추적하기 위해 `struct vma` 를 사용한다
- 이미 `proc.h` 에 제공되어 있으며 `struct proc` 에 `vmas[NVMA]` 로 포함된다

```
struct vma {
    int      used;      // 슬롯 사용 중인지?
    uint64   addr;      // 시작 VA (page-aligned)
    uint64   length;    // 크기 (PGSIZE 배수)
    int      prot;      // PROT_READ / PROT_WRITE
    int      flags;     // MAP_SHARED / MAP_PRIVATE
    int      offset;    // 파일 내 시작 오프셋
    struct file *f;     // filedup으로 refcount 관리
};
```

- mmap 호출 시 fd를 단테라도 매핑이 유지되어야 하므로, VMA에 파일을 등록할 때 반드시 `filedup` 으로 refcount를 올려야 한다
- VMA가 해제될 때 `fileclose` 로 refcount를 내린다

Lazy Allocation 방식

- `mmap` 은 물리 페이지를 즉시 할당하지 않고 VMA의 메타데이터만 기록한다
 - 명세에서 제시된 "records intent"가 이에 해당한다
- 실제 물리 페이지 할당과 파일 데이터 로드는 첫 접근 시 발생하는 page fault 시점에 이루어진다.

```
mmap() 호출
→ VMA 슬롯에 메타데이터만 기록
→ 물리 페이지 할당 없음

user가 mmap 주소 접근
→ page fault 발생
→ mmapfault() 호출
→ 물리 페이지 kalloc
→ 파일에서 read로 데이터 로드
→ mappages로 VA→PA 연결
→ 재실행
```

Implementation

Step 1: `sys_mmap` (`kernel/sysfile.c`)

```
uint64
sys_mmap(void)
{
    int len, prot, flags, fd, offset;
    struct file *f;
    struct proc *p = myproc();

    argint(0, &len);
    argint(1, &prot);
    argint(2, &flags);
    if(argfd(3, &fd, &f) < 0) return -1;
    argint(4, &offset);

    // 인자 검증
    if(len <= 0 || len % PGSIZE != 0) return -1;
    if(offset < 0 || offset % PGSIZE != 0) return -1;
    if((flags & MAP_SHARED) && (prot & PROT_WRITE) && !f->writable)
        return -1;

    // 빈 VMA 슬롯 탐색
    struct vma *v = 0;
    for(int i = 0; i < NVMA; i++){
        if(!p->vmass[i].used){
            v = &p->vmass[i];
            break;
        }
    }
    if(v == 0) return -1;

    // VA 결정: TRAPFRAME 아래, 기존 VMA 중 가장 낮은 주소에서 더 아래로
    uint64 addr = TRAPFRAME;
    for(int i = 0; i < NVMA; i++){
        if(p->vmass[i].used && p->vmass[i].addr < addr)
            addr = p->vmass[i].addr;
    }
    addr -= len;
    if(addr < p->sz) return -1; // heap과 겹침

    // VMA 등록
    v->used = 1;
```

```

v->addr = addr;
v->length = len;
v->prot = prot;
v->flags = flags;
v->offset = offset;
v->f = filedup(f); // refcount 증가 필수

return addr;
}

```

- `sys_mmap` 은 물리 페이지를 전혀 건드리지 않고 VMA 메타데이터만 채운다

```

if((flags & MAP_SHARED) && (prot & PROT_WRITE) && !f->writable)
    return -1;

```

- 이 조건은 다음의 조건이 같이 발생할 때를 방지한다
 - `(flags & MAP_SHARED)` : unmap시 수정이 파일에 반영 돼야 함
 - `(prot & PROT_WRITE)` : 쓰기 권한으로 요청됨
 - `!f->writable` : 파일이 쓰기가 불가능한 경우임
 - 파일을 open할 때 읽기 전용으로 열었을 때, 이 조건들에 대한 검사가 없다면, 파일을 수정할 수 있다
- mmap 주소는 kernel이 결정하며, 기존 VMA 중 가장 낮은 주소에서 `len` 만큼 더 아래로 내려가는 방식으로 겹침을 방지한다
- `filedup` 으로 refcount를 올려 fd를 나중에 닫아도 VMA가 유지되게 한다

Step 2: `mmapfault` (`kernel/sysfile.c`)

```

int
mmapfault(uint64 va)
{
    struct proc *p = myproc();
    va = PGROUNDDOWN(va);

    // va를 포함하는 VMA 탐색
    struct vma *v = 0;
    for(int i = 0; i < NVMA; i++)
        if(p->vmass[i].used && va >= p->vmass[i].addr &&
            va < p->vmass[i].addr + p->vmass[i].length){
            v = &p->vmass[i]; break;
        }
    if(v == 0) return -1;

    // 이미 매핑된 페이지: mappages: remap 방지
    if(walkaddr(p->pagetable, va) != 0) return -1;

    // 물리 페이지 할당
    char *mem = kalloc();
    if(mem == 0) return -1;
    memset(mem, 0, PGSIZE);

    // 파일에서 데이터 읽기
    uint64 foff = v->offset + (va - v->addr);
    ilock(v->f->ip);
    readi(v->f->ip, 0, (uint64)mem, foff, PGSIZE); // short read는 무시
    iunlock(v->f->ip);

    // prot → PTE 권한 변환 (PROT_* ≠ PTE_*)
    int perm = PTE_U;

```

```

if(v->prot & PROT_READ) perm |= PTE_R;
if(v->prot & PROT_WRITE) perm |= PTE_W;

// VA → PA 매핑
if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) < 0){
    kfree(mem);
    return -1;
}

return 0; // 성공
}

```

- `readi` 의 두 번째 인자 `0` 은 `user_dst=0` 으로, kernel 주소에 직접 쓴다는 의미이다
- EOF 근처에서 short read가 발생해도 나머지는 `memset` 으로 이미 0으로 채워져 있으므로 무시해도 안전하다
- `PROT_READ` 와 `PTE_R` 은 값이 다르므로 반드시 명시적으로 변환해야 한다
- 두 번째 fault(이미 매핑된 페이지)에서는 -1을 반환하여 `mappages: remap` panic을 방지한다

Step 3: `sys_munmap` (`kernel/sysfile.c`)

```

uint64
sys_munmap(void)
{
    uint64 addr; int len;
    argaddr(0, &addr);
    argint(1, &len);

    if(len <= 0 || len % PGSIZE != 0 || addr % PGSIZE != 0) return -1;

    struct proc *p = myproc();
    struct vma *v = 0;
    for(int i = 0; i < NVMA; i++)
        if(p->vmass[i].used && addr >= p->vmass[i].addr &&
            addr < p->vmass[i].addr + p->vmass[i].length){
            v = &p->vmass[i]; break;
        }
    if(v == 0) return -1;

    // middle-hole 거부
    if(addr != v->addr && addr + len != v->addr + v->length) return -1;
    if(addr + len > v->addr + v->length) return -1;

    // MAP_SHARED: 수정된 페이지를 파일에 write-back
    if(v->flags & MAP_SHARED){
        for(uint64 a = addr; a < addr + len; a += PGSIZE){
            if(walkaddr(p->pagetable, a) == 0) continue; // 한 번도 안 쓴 페이지 스킵
            uint64 foff = v->offset + (a - v->addr);
            begin_op();
            ilock(v->f->ip);
            writei(v->f->ip, 1, a, foff, PGSIZE);
            iunlock(v->f->ip);
            end_op();
        }
    }

    // 물리 페이지 해제 + PTE 클리어
    vmunmap(p->pagetable, addr, len / PGSIZE, 1);

    // VMA 범위 조정

```

```

if(addr == v->addr){
    v->offset += len;
    v->addr += len;
    v->length -= len;
} else {
    v->length -= len;
}

// VMA 전체 해제
if(v->length == 0){
    fclose(v->f);
    v->used = 0;
    v->f = 0;
}

return 0;
}

```

- `walkaddr` 가 0을 반환하는 페이지는 한 번도 fault가 나지 않은(lazy 상태) 페이지이므로 write-back을 건너뛰다
- `begin_op` / `end_op` 는 `writei` 가 파일 시스템 로그를 사용하기 때문에 필수다.
- `munmap`은 앞부분 또는 끝부분 제거만 허용하고, 가운데 구멍(middle-hole)은 -1로 거부한다

Step 4: `kfork` / `kexit` hooks (`kernel/proc.c`)

`kfork` : VMA 복사

- `for(i=0; i<NOFILE; i++) filedup(...)` 바로 다음에 추가했다

```

// VMA 메타데이터 복사 + 파일 refcount 증가
// 물리 페이지는 공유하지 않음 → 자식이 lazy fault로 직접 로드
for(int i = 0; i < NVMA; i++){
    if(p->vmass[i].used){
        np->vmass[i] = p->vmass[i];
        filedup(p->vmass[i].f); // refcount 증가 필수
    }
}

```

- 자식은 부모와 같은 VMA 메타데이터를 갖지만 물리 페이지는 공유하지 않는다
- 자식이 `mmap` 영역에 처음 접근할 때 `mmapfault` 가 물리 페이지를 독립적으로 할당한다
- `filedup` 을 호출하지 않으면 자식과 부모 각자가 `fclose` 를 호출할 때 `refcount`가 1 미만으로 떨어져 `panic: fclose` 가 발생한다

`kexit` :VMA write-back + 해제

- 파일 닫기 루프(`ofile[]` 정리) 직후, `begin_op()/iput(p->cwd)` 직전에 추가했다

```

// pagetable[] 해제되기 전에 실행해야 함
for(int i = 0; i < NVMA; i++){
    if(p->vmass[i].used){
        struct vma *v = &p->vmass[i];
        if(v->flags & MAP_SHARED){
            for(uint64 a = v->addr; a < v->addr + v->length; a += PGSIZE){
                if(walkaddr(p->pagetable, a) == 0) continue;
                uint64 foff = v->offset + (a - v->addr);
                begin_op();
                ilock(v->f->ip);
                writei(v->f->ip, 1, a, foff, PGSIZE);
                iunlock(v->f->ip);
                end_op();
            }
        }
    }
}

```

```

    }
    uvmunmap(p->pagetable, v->addr, v->length / PGSIZE, 1);
    fclose(v->f);
    v->used = 0;
    v->f = 0;
}
}

```

- `proc_freepagetable` (pagetable 해제)보다 먼저 실행해야 `walkaddr`와 `uvmunmap`이 유효한 pagetable을 사용할 수 있다
- `kexit`에서도 `begin_op` / `end_op`가 반드시 필요하다

Results

mmaptest pass

```

inbeomjin@inbeomjin-ui-MacBookAir ~/project3-Ben9960 1 main ± make qemu
hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
             sepc=0xaae stval=0x3ffffd000
usertrap(): unexpected scause 0xd pid=6
             sepc=0xb28 stval=0x3ffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
             sepc=0xc80 stval=0x3ffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded

```

- 두 테스트 모두 OK로 통과한다.

usertests -q 전체 통과

```

PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
$ 

```

- `mmap`/`munmap` 구현이 기존 `xv6`의 동작을 깨뜨리지 않았음을 확인했다.

Troubleshooting

1. heap 겹침 조건 오류

```

// 초기 구현
if(addr == p->sz) return -1; // 정확히 같을 때만 거부

```

```
// 수정
if(addr < p->sz) return -1;    // heap 이하면 거부
```

- == 만 검사하면 addr 이 p->sz 보다 훨씬 낮은 경우(heap 한가운데 충돌)를 잡지 못한다
- < 로 수정해야 heap 전체 범위를 보호한다.

2. prot 비트 검사: == 대신 &

```
// ❌ 초기 구현
if(v->prot == PROT_READ) perm |= PTE_R;
if(v->prot == PROT_WRITE) perm |= PTE_W;
```

- == 는 prot가 정확히 그 값 하나일 때만 참이다.
- PROT_READ | PROT_WRITE 처럼 두 플래그가 함께 있는 경우 둘 다 거짓이 되어 PTE 권한이 아무것도 설정되지 않았다

```
// 수정
if(v->prot & PROT_READ) perm |= PTE_R;
if(v->prot & PROT_WRITE) perm |= PTE_W;
```

3. kfork 에서 filedup 누락 → panic: fileclose

- test fork 단계에서 다음과 같은 panic이 발생했다

```
panic: fileclose
```

- fileclose 는 f->ref < 1 일 때 panic한다
- 원인은 kfork에서 VMA를 복사할 때 filedup 을 호출하지 않아 refcount가 균형을 잃은 것이다
- refcount 흐름 (filedup 누락 시):

```
open(fd):          ref = 1
sys_mmap filedup:  ref = 2
kfork ofile 복사:  ref = 3 (filedup 자동 호출)
kfork VMA 복사:    ref = 3 (filedup 없음! ← 버그)

자식 kexit:
  ofile fileclose: ref = 2
  VMA fileclose:   ref = 1

부모:
  ofile fileclose: ref = 0 → 해제됨
  VMA fileclose:   ref = -1 → panic!
```

```
// 수정: kfork에서 반드시 filedup 호출
for(int i = 0; i < NVMA; i++){
    if(p->vmass[i].used){
        np->vmass[i] = p->vmass[i];
        filedup(p->vmass[i].f);    // ← 이 줄이 없으면 panic
    }
}
```